Лабораторная работа № 2 по курсу дискретного анализа: сбалансированные деревья

Выполнил студент группы М8О-203БВ-24 *Пятницкий Артём*.

Условие

1. Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64} - 1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

- **+ word 34** — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.
- **- word** — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.
- **word** — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».
- **! Save /path/to/file** — сохранить словарь в бинарном компактном представлении на диск в файл, указанный парамером команды. В случае успеха, программа должна вывести «OK», в случае неудачи выполнения операции, программа должна вывести описание ошибки (см. ниже).
- **! Load /path/to/file** — загрузить словарь из файла. Предполагается, что файл был ранее подготовлен при помощи команды Save. В случае успеха, программа должна вывести строку «OK», а загруженный словарь должен заменить текущий (с которым происходит работа); в случае неуспеха, должна быть выведена диагностика, а рабочий словарь должен остаться без изменений. Кроме системных ошибок, программа должна корректно обрабатывать случаи несовпадения формата указанного файла и представления данных словаря во внешнем файле.

Для всех операций, в случае возникновения системной ошибки (нехватка памяти, отсутсвие прав записи и т.п.), программа должна вывести строку, начинающуюся с «ERROR:» и описывающую на английском языке возникшую ошибку.

Вариант 3. 3 PATRICIA:

PATRICIA.

Ограничение по времени	15 секунды
Ограничение по памяти	512 Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Пример:

Ввод	**Вывод**
+ a 1 + A 2 + aaaaa 18446744073709551615 aaaaa A - A a	OK Exist OK OK: 18446744073709551615 OK: 1 OK NoSuchWord

Метод решения

Для решения данной задачи было необходимо для начала определить структуру узла. Узел представляет из себя шаблонный класс, с полями:

«Left», «Right», «index», «key», «value». Значение «Value» представляет из себя скалярный тип «T». Все изменения в узле происходят через публичные методы получения приватных полей. Само дерево – класс, который хранит в себе единственный атрибут – «root». Корень представляет из себя узел, который не имеет правого ребёнка, хранит в себе первое вставленное значение и всегда имеет индекс 0.

Для сравнения двух ключей используются битовые операции, которые позволяют ускорить вычисления сэкономить память. Функция использует операцию «XOR» для нахождения первого различающегося бита в двух ключах. Также присутствует функция нахождения бита в ключе по индексу. Она получает конкретный индекс и возвращает значение бита по этому индексу. Нумерация индексов в ключах начинается с 1, кроме корня, который имеет индекс 0. Нумерация битов в функции идёт слева направо, начиная с 0:

Индексы битов	Индексы Node
Идут слева направо начиная с 0	От 1 (Кроме корня, у него 0)
0 1 2 3 4 5 6 7	До 256 * 8

Для вычисления индекса для начала происходит вычисление индекса элемента в ключе (индекс символа в строке), а потом вычисляется индекс бита в символе. Нахождение значения конкретного бита использует побитовое "И" с маской. Мы берем единичную маску 00000001 и сдвигаем её влево на расчетный индекс. Поскольку в ключе биты нумеруются слева направо (от старшего к младшему), индекс бита внутри байта вычисляется как 7 - ((индекс – 1) % 8)

Рассмотрим работу функции на простом примере:
В качестве ключа передаётся строка «abc», в качестве индекса передаётся число 15.

Расчёт для индекса 15 (ключ «abc»)

Параметр	**Формула / Значение**	**Результат**
Общий индекс	index	15
Индекс символа	byte_index = (index - 1) / 8	1 (символ 'b')
Код символа (bin)	code = ASCII 'b'	01100010
Индекс бита	bit_index = 7 - ((15 - 1) % 8)	1
Маска (bin)	mask = 1 << bit_idx	00000010
Операция побитового «И»	code & mask	00000010
Значение бита в индексе	(результат != 0) ? 1 : 0	1

Алгоритм вставки выполняет поиск, а затем вставляет новый узел на его позицию. Поиск выполняется путём прохода по дереву, пока не происходит переход по обратной ссылке. Поиск всегда начинается с корня, и спускается влево, так как у корня нет правого ребёнка. Если у ключа, который мы вставляем на индексе текущего узла стоит 1, то спускаемся вправо, если 0, то

влево. Спуск происходит до тех пор, пока индекс текущего узла больше индекса предыдущего. Если мы перешли по обратной ссылке, мы сравниваем ключи (Если такой ключ уже есть, элемент с таким ключом уже существует и вставлять ничего не надо). С помощью функции «get_first_bit_diff» находим первый различающийся бит в двух ключах – это индекс нового узла. Теперь мы должны найти место, куда мы можем вставить наш новый узел, поэтому снова выполняем спуск по дереву, но помимо сравнения индекса текущего и предыдущего узлов, сравниваем также новый индекс с текущем (Если новый индекс больше текущего, продолжаем поиск, иначе останавливаем поиск). Как только поиск закончился, необходимо вставить новый узел на его новое место. Находим значение бита в нашем ключе по индексу вставляемого узла, дальше необходимо задать связи у нового узла:

Значение бита	Левый сын	Правый сын
0	Ссылка на себя	Текущий узел после поиска
1	Текущий узел после поиска	Ссылка на себя

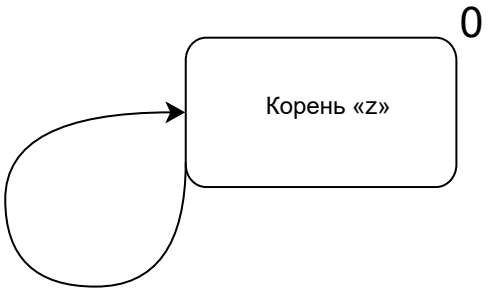
Также надо поменять ссылки у предыдущего узла, чтобы он ссылался на нас. Находим значение бита в нашем ключу по индексу предыдущего узла:

Значение бита	Левый сын предыдущего	Правый сын Предыдущего
0	Ссылка на новый узел	-
1	-	Ссылка на новый узел

Ниже приведён алгоритм вставки на примере вставки двух элементов:

Ключ (буква)	Бинарный код	Индекс
z	01111010	0
p	01110000	-

Начнём вставлять наши буквы по порядку. Вставляем первую букву «z», так как это корень её индекс всегда 0. Правой ссылки у корня нет, а левая всегда указывает сама на себя:



Теперь вставим букву «p». Начинаем поиск (0 → 0). Перешли по обратной ссылке:

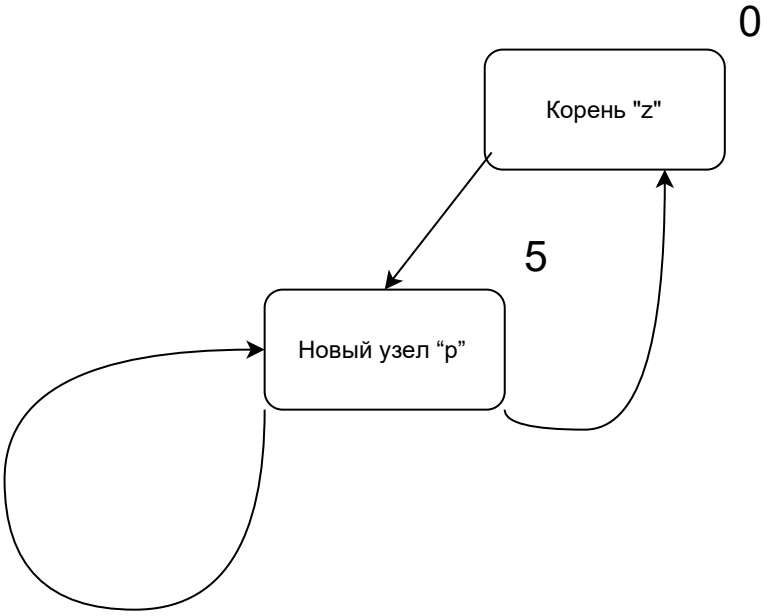
Текущий	Предыдущий
Корень	Корень

Так как это корень мы можем вставить только слева. Индекс, в котором различается «Z» и «P» равен 5, следовательно у узла индекс будет равен 5

Ключ (буква)	Бинарный код	Индекс
z	01111010	0
p	01110000	5

Теперь надо создать ссылки на детей у нового узла:

В 5 индексе буква «P» имеет бит 0, следовательно левая ссылка указывает сама на себя, а правая указывает на корень (текущий узел):



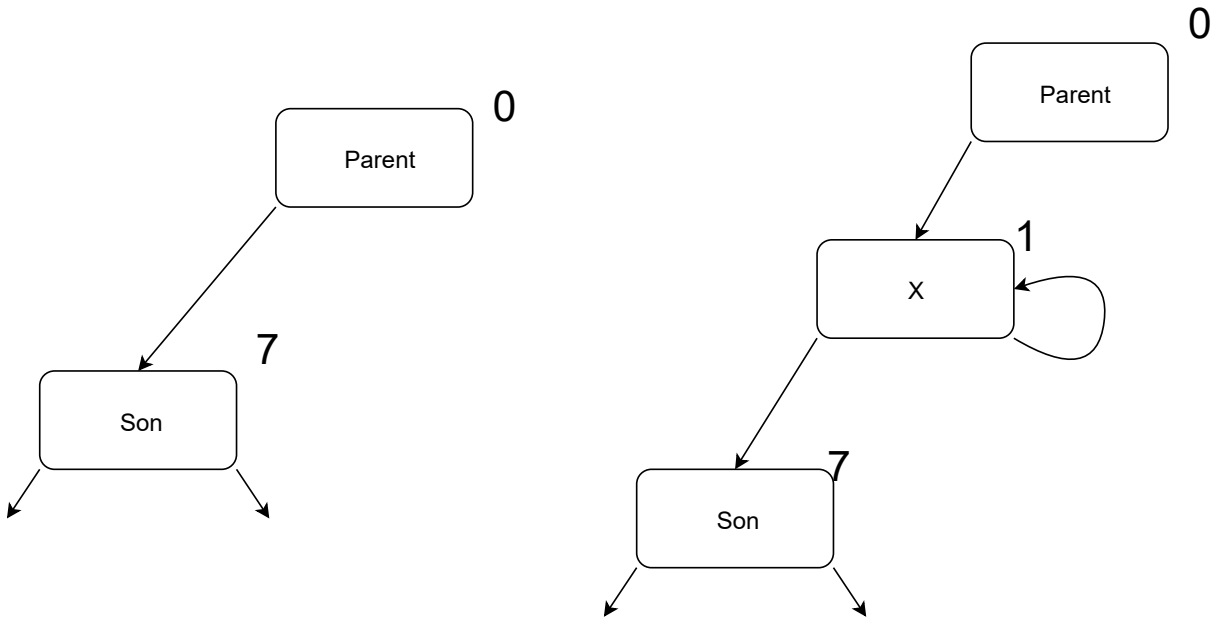
Таким образом мы успешно добавили элемент.

Алгоритм поиска представляет из себя простой обход по дереву, до тех пор, пока не произойдёт переход по обратной ссылке. После перехода по обратной ссылке происходит сравнение элементов. Если ключи не совпали, то элемента нет.

Алгоритм удаления является самым сложным, как в реализации, так и с точки зрения понимания. Для удаления рассматриваются 3 случая:

0. Удаление одного элемента – корня (самый простой случай, когда мы просто удаляем 1 элемент).
1. Удаление элемента, у которого обратная ссылка ссылается на самого себя.
2. Удаление элемента, у которого обратная ссылка идёт не на себя.

Рассмотрим случай 1. Этот случай происходит, когда удаляемый ключ хранится в узле «X», и одна из его веток является **обратной ссылкой**, указывающей на этот же узел «X». Для удаления такого элемента необходимо найти его родителя и его сына (это та ветка, которая НЕ указывает на «X». Она может вести как к другому внутреннему узлу, так и быть обратной ссылкой на предка выше). Затем нужно заменить связь родителя с удаляемым элементом «X» на прямую связь с этим сыном.



Рассмотрим случай 2. Этот случай сложнее. Он происходит, когда мы удаляем ключ, на который указывает обратная ссылка из **другого** (более глубокого) узла дерева. То есть узел «X», который мы нашли при поиске, не ссылается сам на себя, а на него ссылается какой-то узел «Q», находящийся

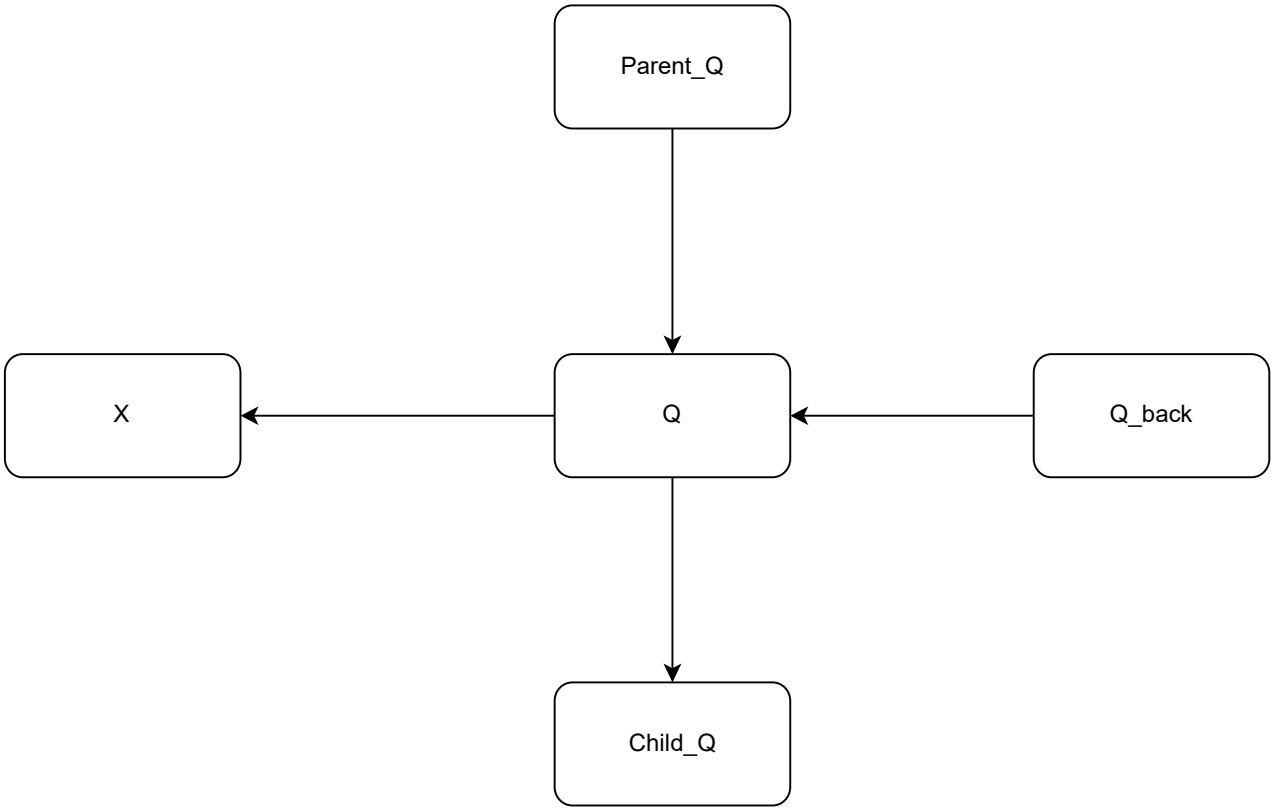
ниже по дереву. В такой ситуации мы будем удалять узел «Q», а вместо удаления узла «X», просто поместим в него значения ключа и значения из узла «Q», который будем удалять на самом деле.

Процесс изоляции узла – такой процесс преобразования дерева, который позволяет найти любой элемент в дереве, без этого узла. Чтобы нам удалить «Q», мы должны изолировать его, а потом спокойной удалить. Для этого нам понадобится предок элемента «Q» - «Parent_Q», элемент, который ссылается на «Q» по обратной ссылке – «Q_back», и ребёнок элемента «Q» - «Q_child», а также сам элемент «X» . Таким образом у нас есть:

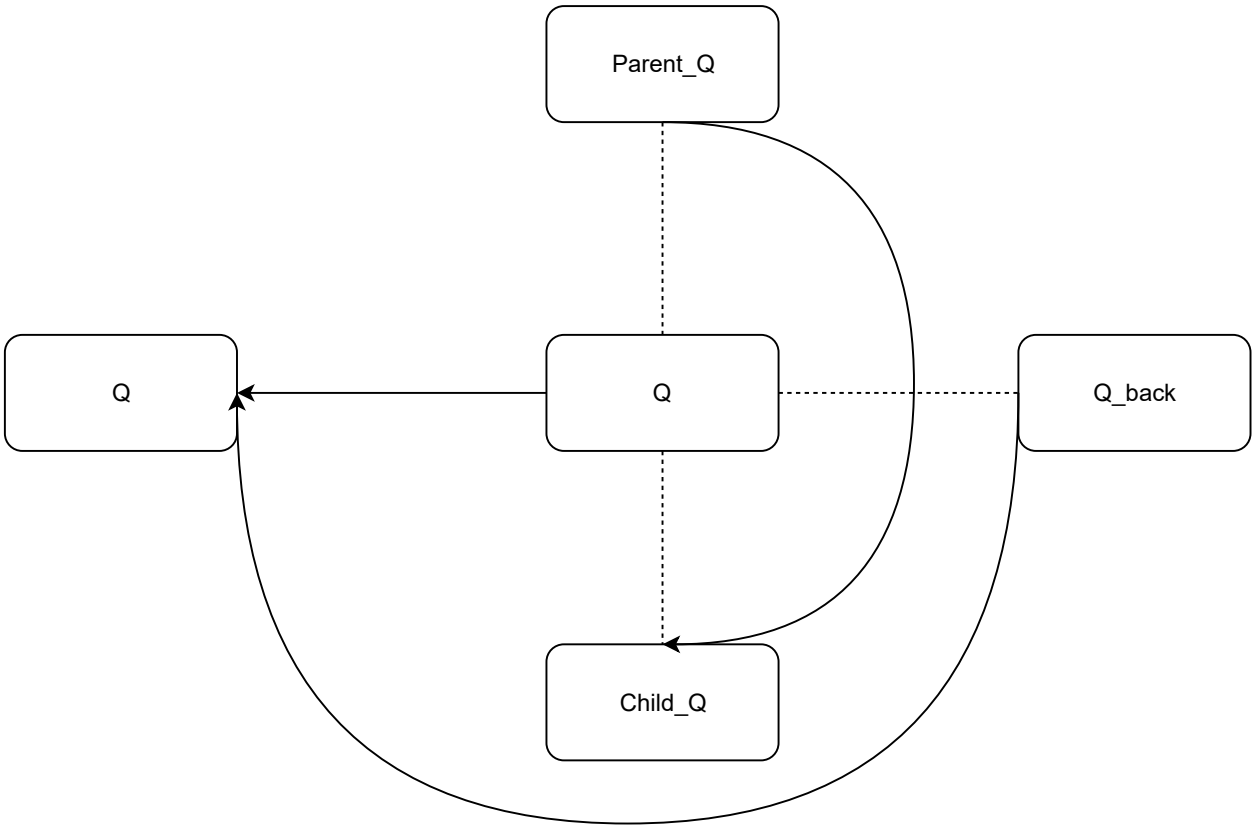
Элемент, который хотим удалить	Элемент, который ссылается на «X», по обратной ссылке	Элемент, который ссылается на «Q», по обратной ссылке	Родитель элемента «Q»	Ребёнок «Q»
X	Q	Q_back	Parent_Q	Child_Q

Для удаления мы сделаем следующее: «Q_back» будет ссылаться на «X». А элемент «Parent_Q» по связи к «Q» будет связан с элементом «Child_Q».

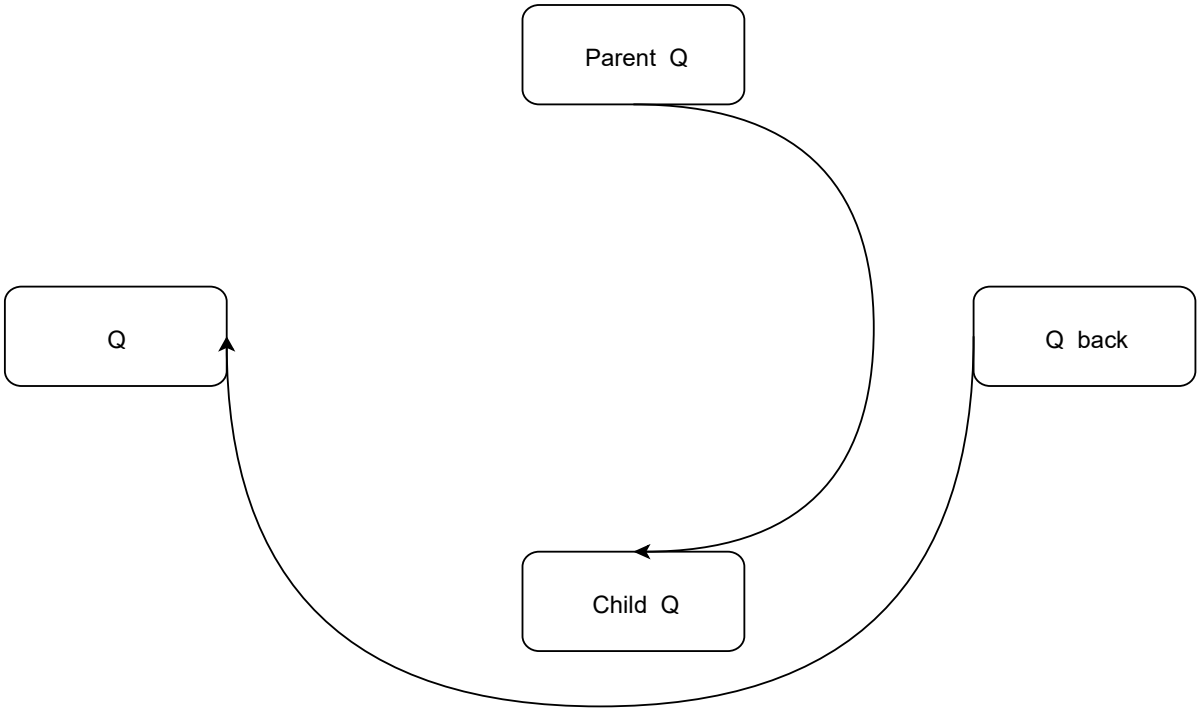
Исходная ситуация:



Этап изоляции узла «Q» и переноса ключа и значения в элемент «X»:



Удаляем узел «Q»:



Логика сохранения и загрузки дерева реализована через последовательную запись пар «ключ-значение» в бинарный файл. Данный подход обоснован тем, что структура дерева PATRICIA

является **детерминированной относительно набора ключей**: итоговый вид дерева зависит только от состава данных, а не от порядка их добавления. Таким образом, при повторном построении дерева из файла мы гарантированно получаем структуру, идентичную исходной.

Примечание: в данной реализации корень является базовым элементом, относительно которого строится дальнейшая топология дерева. Чтобы гарантированно восстановить **идентичную** структуру (с тем же самым корневым узлом), запись в файл всегда начинается именно с него. Таким образом, при загрузке первым считывается и вставляется исходный корень, что обеспечивает полное соответствие восстановленного дерева оригиналу.

Описание программы

Разделение по файлам, описание основных типов данных и функций.

Класс узла:

```
template<typename T>
concept Scalar = std::is_scalar_v<T>;

template<Scalar T>
class Node {
    private:
        std::string key;
        T value;
        size_t index;
        Node* left = nullptr;
        Node* right = nullptr;
    public:
        Node(const std::string& key, const T& value, const size_t& index)
        : key(key), value(value), index(index) {}
        // Стандартный деструктор
        ~Node() = default;
        const std::string& get_key() {return key;}
        T& get_value() {return value;}
        const T& get_value() const {return value;}
        size_t get_index() {return index;}
        void set_value(const T& new_value) {value = new_value;}
        void set_key(const std::string& new_key) {key = new_key;}
        void set_left(Node<T>* node) {left = node;}
        void set_right(Node<T>* node) {right = node;}
        Node<T>* get_left() {return left;}
        Node<T>* get_right() {return right;}
};
```

Методы для поиска бита (get_bit и get_first_bit_diff):

```
uint8_t get_bit(const std::string& key, size_t index) {
            if (index == 0) return 0;
            size_t byte_index = (index - 1) / 8;
            if(byte_index >= key.length()) return 0;
            uint8_t mask = 1;
            uint8_t byte = key[byte_index];
            // Считаем, что мы идём по байту слева направо, тогда наши индексы будут
располагаться так:
            // 11111111 -> 01234567, где 0 - самый старший бит, а 7 - самый младший
            mask = (1 << (7 - (index - 1) % 8));
            return ((byte & mask) == 0 ? 0 : 1);
        }
        size_t get_first_bit_diff(const std::string& key1, const std::string& key2) {
```

```
            size_t len_key1 = key1.length(), len_key2 = key2.length();
            size_t n = std::max(len_key1, len_key2);
            for (size_t i = 0; i < n; ++i) {
                uint8_t c1 = (i < len_key1) ? static_cast<uint8_t>(key1[i]) : 0;
                uint8_t c2 = (i < len_key2) ? static_cast<uint8_t>(key2[i]) : 0;
                if (c1 != c2) {
                    uint8_t diff = c1 ^ c2;
                    size_t position = 7;
                    while (!(diff & (1 << position))) {
                        --position;
                    }
                    return i * 8 + (7 - position) + 1;
                }
            }
            return 0;
        }
```

Методы вставки, поиска и удаления:

```
bool insert(const std::string& key, const T& data) {
            if (root == nullptr) {
                root = new Node<T>(key, data, 0);
                root->set_left(root);
                return true;}
            Node<T>* prev = root;
            Node<T>* current = root->get_left();
            while(current->get_index() > prev->get_index()) {
                prev = current;
                uint8_t bit = get_bit(key, current->get_index());
                current = (bit == 0) ? current->get_left() : current->get_right();}
            if (key == current->get_key()) {return false;}
            size_t bit_diff_index = get_first_bit_diff(key, current->get_key());
            prev = root;
            current = root->get_left();
            while (current->get_index() > prev->get_index() && current->get_index() <
bit_diff_index) {
                prev = current;
                uint8_t bit = get_bit(key, current->get_index());
                current = (bit == 0) ? current->get_left() : current->get_right();
            }
            Node<T>* newNode = new Node<T>(key, data, bit_diff_index);
            uint8_t bit = get_bit(key, bit_diff_index);
            newNode->set_left(bit == 0 ? newNode : current);
            newNode->set_right(bit == 0 ? current : newNode);
            if (prev == root) {
                prev->set_left(newNode);}
            else {
                uint8_t prevbit = get_bit(key, prev->get_index());
                if (prevbit == 1) prev->set_right(newNode);
                else prev->set_left(newNode);}
            return true;
        }
        T* search(const std::string& key) {
            if (root == nullptr) return nullptr;
            Node<T>* current = root->get_left();
            Node<T>* prev = root;
            while (current->get_index() > prev->get_index()) {
                prev = current;
                uint8_t bit = get_bit(key, current->get_index());
                current = bit == 0 ? current->get_left() : current->get_right();
            }if (key == current->get_key()) {
                return &(current->get_value());}
            else {return nullptr;}
        }
        bool remove(const std::string& key) {
            if (root == nullptr) return false;
```

```
        // Удаляем корень без ничего
        if (key == root->get_key() && root->get_left() == root) {
            delete root;
            root = nullptr;
            return true;}
        // Случай 1. Удаляем элемент, который имеет обратную ссылку на себя
        Node<T>* Target = nullptr;
        Node<T>* q = nullptr;
        Node<T>* parent_q = nullptr;
        search_target_q(key, Target, q, parent_q);
        if (Target == nullptr) return false;
        if (Target == q && Target != root) {
            Node<T>* child = nullptr;
            if (Target->get_left() == Target) child = Target->get_right();
            else if (Target->get_right() == Target) child = Target->get_left();
            if (child != nullptr) {
                uint8_t bit = get_bit(key, parent_q->get_index());
                if (bit == 0) parent_q->set_left(child);
                else parent_q->set_right(child);
                delete Target;
                return true;
            }
        }
        // Случай 2. Удаляем элемент, который не имеет обратной ссылки на себя
        Node<T>* tempTg = nullptr;
        Node<T>* q_back = nullptr;
        Node<T>* tempPr = nullptr;
        Node<T>* child_q = nullptr;
        search_target_q(q->get_key(), tempTg, q_back, tempPr);
        if (Target == q->get_left()) child_q = q->get_right();
        else child_q = q->get_left();
        if (child_q == q) {
            child_q = Target;
        }
        if (parent_q != nullptr) {
            uint8_t bit = get_bit(key, parent_q->get_index());
            if (bit == 0) parent_q->set_left(child_q);
            else parent_q->set_right(child_q);}
        else {root->set_left(child_q);}
        if (q_back != nullptr && q_back != q) {
            uint8_t bit = get_bit(q->get_key(), q_back->get_index());
            if (bit == 0) q_back->set_left(Target);
            else q_back->set_right(Target);}
        Target->set_value(q->get_value());
        Target->set_key(q->get_key());
        delete q;
        return true;}
```

Дневник отладки

В ходе выполнения лабораторной работы, возникла проблема с удалением элементов, поскольку было необходимо находить множество узлов, которые потом необходимо было правильно связать между собой, особенно случай «2», когда «Q_back» ссылался сам на себя, в момент тестирования возникала проблема segmentation_fault, которая была вызвана тем, что в момент удаления элемент «X» начинал ссылаться на удалённый участок памяти, но в конечном счёте, ошибка была устранена. Также была проделана большая работа по разбору случаев удаления и вставки, благодаря которой появилось чёткое понимание структуры PATRICIA.

Также одной из проблем была функция записи в файл значения узла.

Так как узел, да и сама структура – шаблонная, проблема заключалась в том, что просто так сохранить значение в файл было нельзя, так как сложные структуры по типу std::vector, std::string и т. д. нельзя так просто записывать в бинарный файл. Для таких структур сначала записывают число элементов в них, а потом их элементы для того, чтобы можно было успешно их потом восстановить. Поэтому было принято решение ограничить тип значения скалярным типом. С помощью concept было создано ограничение std::is_scalar_v<T>, которое гарантирует, что тип значения будет скалярным, что позволяет нам спокойно загружать файл такого рода типы, так как они имеют фиксированный размер в байтах.

Тест производительности

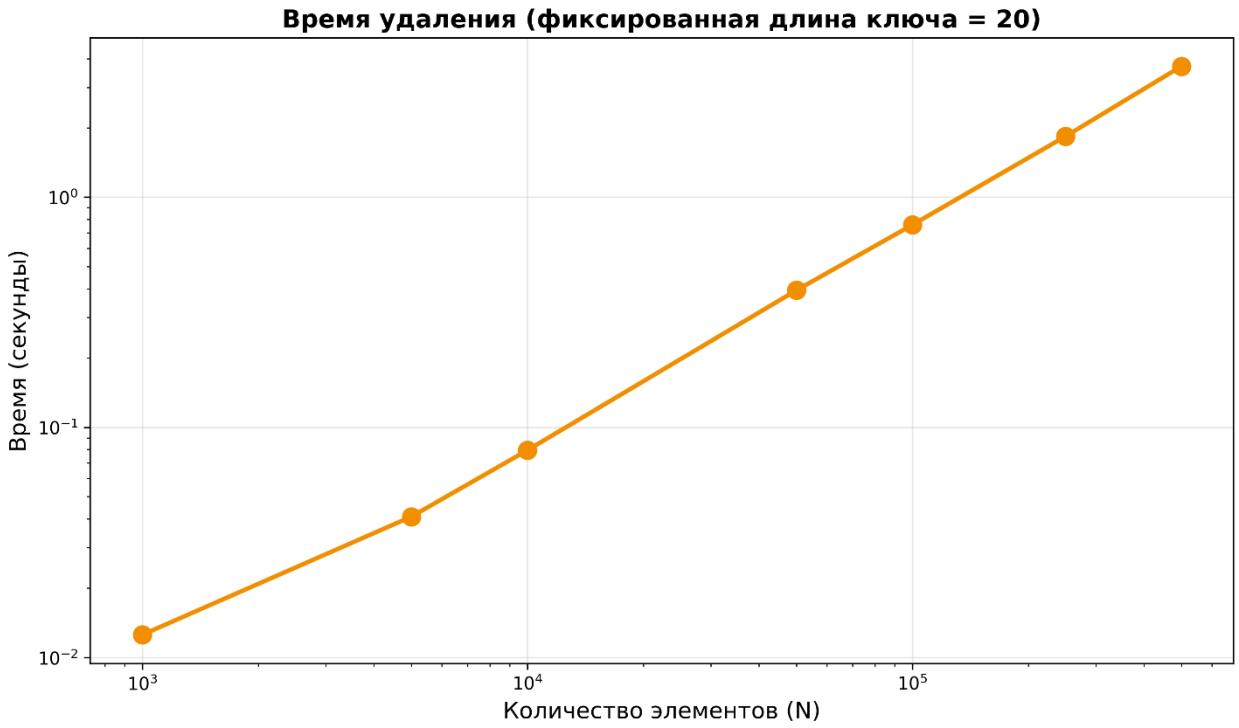
Для проверки скорости работы программы, проведём замер, постепенно увеличивая количество данных и проведём замеры скорости выполнения и потребления памяти:

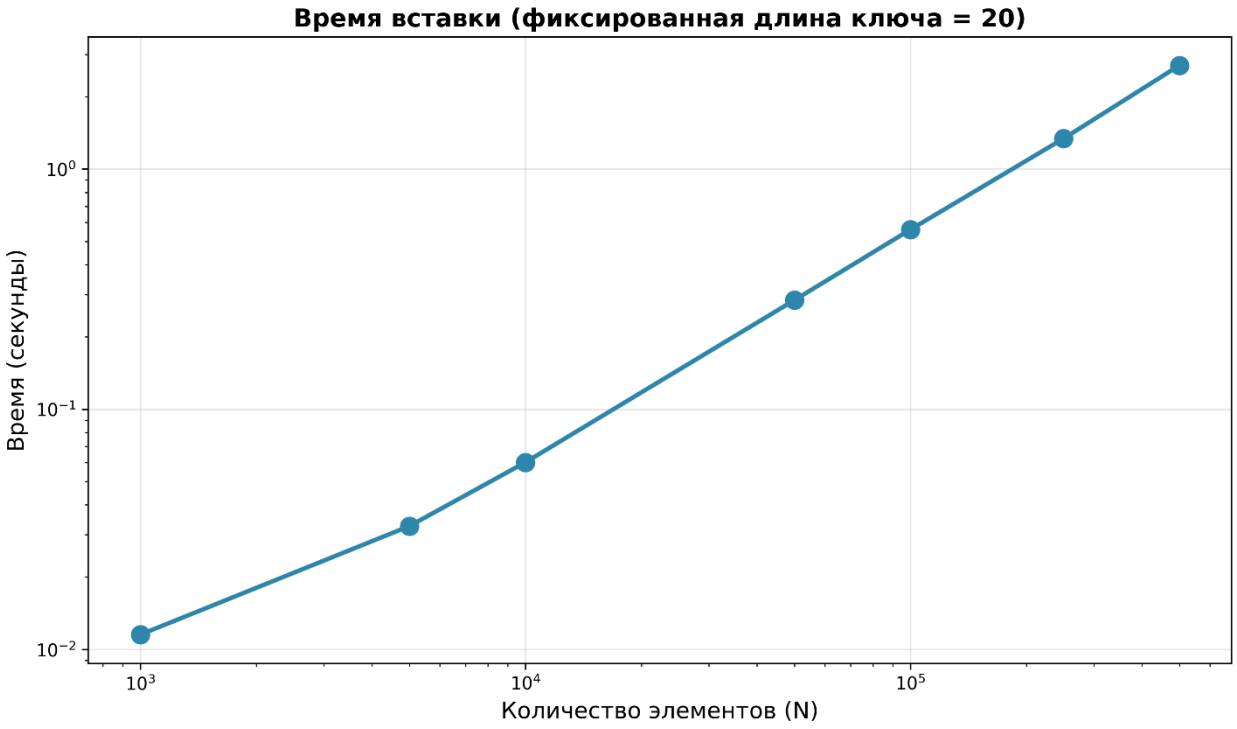
Замер проводился на WSL (Windows Subsystem for Linux)

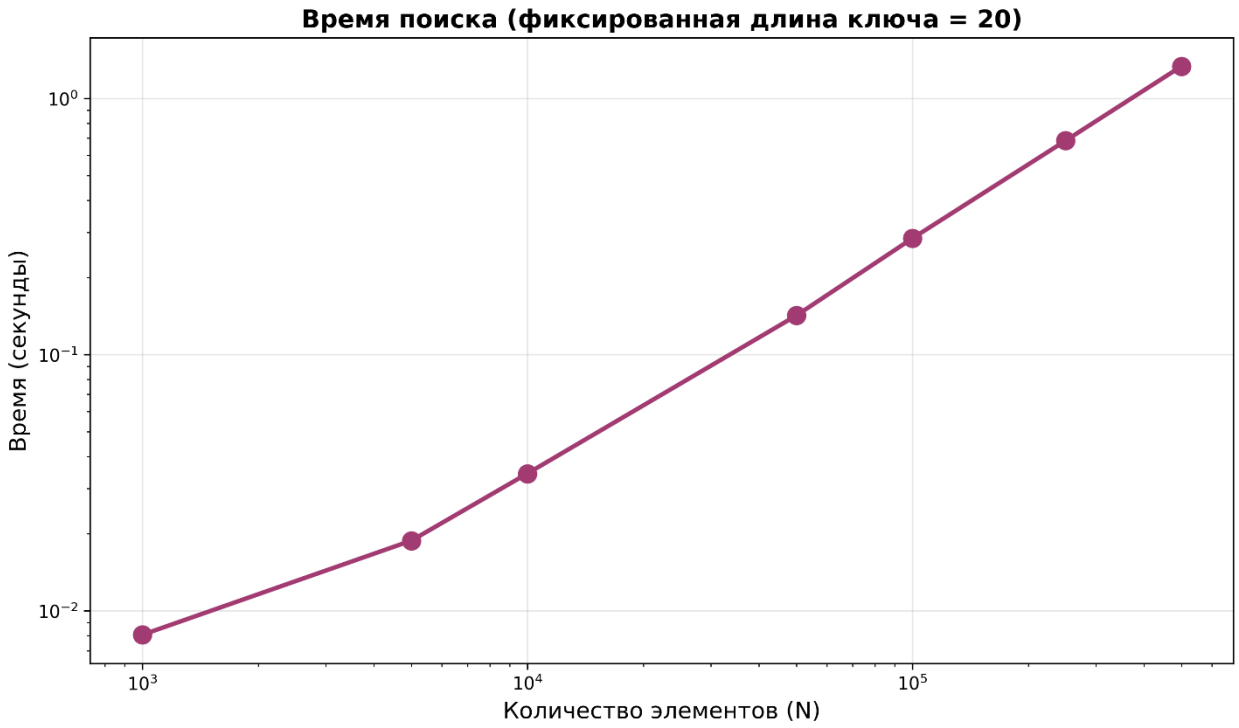
Первый замер проводился с помощью постепенной нагрузки дерева числом элементов начиная от 1000 до 500000 элементов с фиксированной длиной ключа и замером скорости вставки, удаления и писка.

Для измерения времени поиска и удаления используется первоначальная загрузка дерева из файла и дальнейший тест функций. Время измеряется через команду «time».

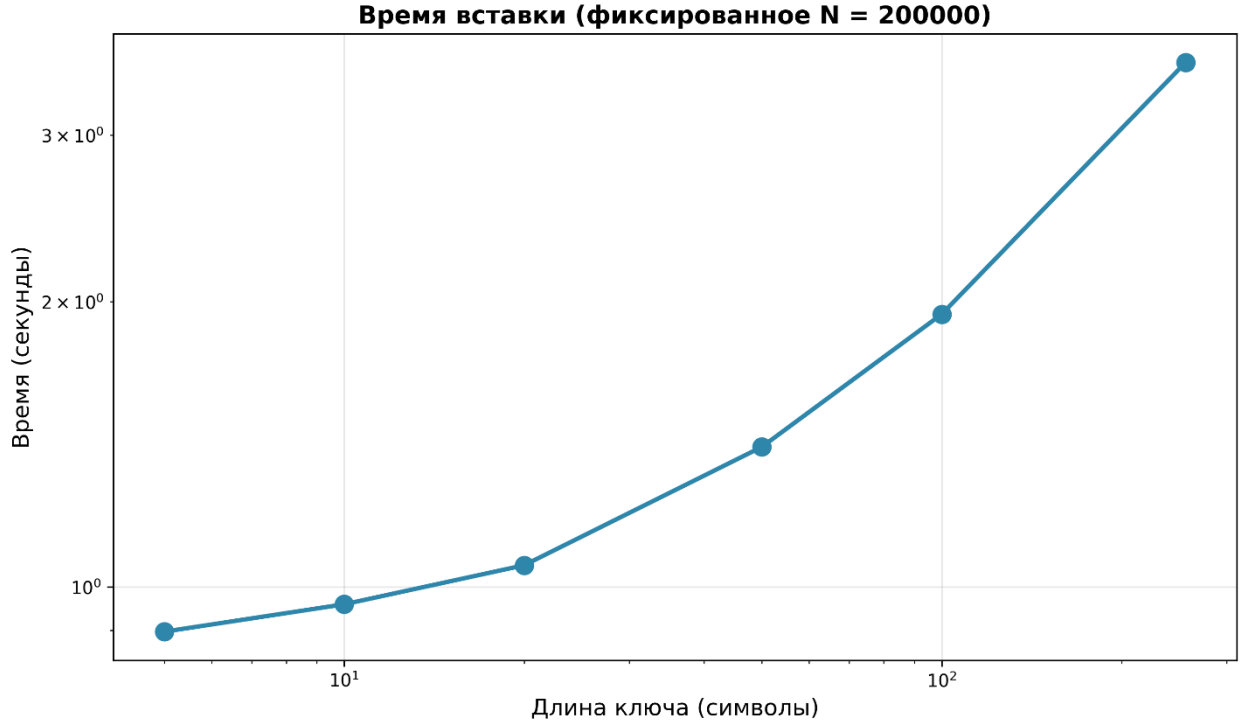
Ниже приведены графики зависимости скорости вставки, поиска и удаления от числа элементов:

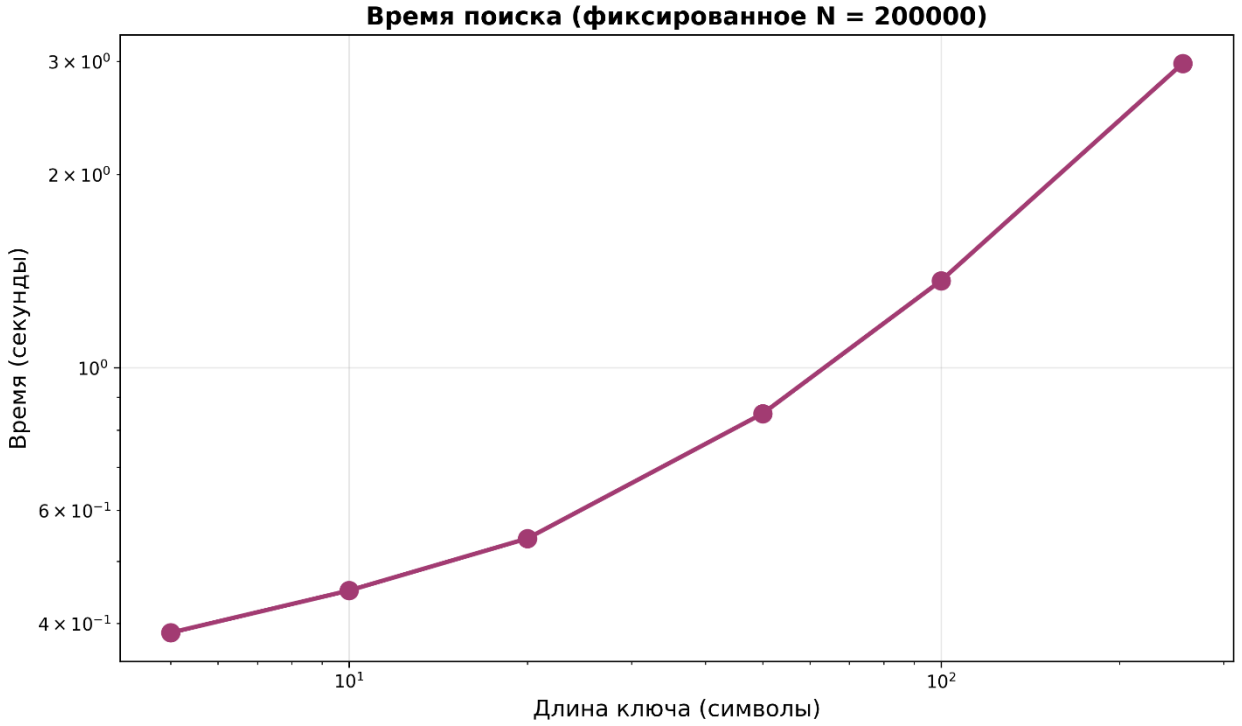

Время удаления (фиксированная длина ключа = 20)
Время (секунды)
10^0
10^{-1}
10^{-2}
10^3
10^4
10^5
Количество элементов (N)


Время вставки (фиксированная длина ключа = 20)
Время (секунды)
10^0
10^{-1}
10^{-2}
10^3
10^4
10^5
Количество элементов (N)

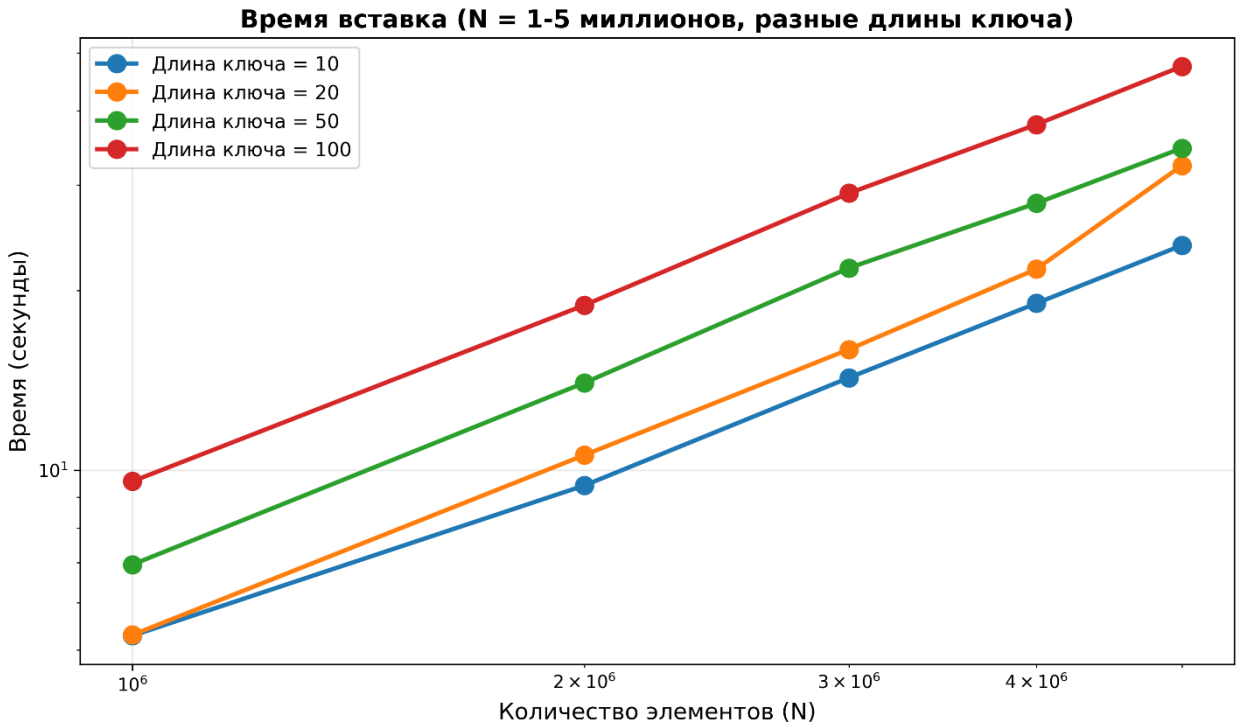


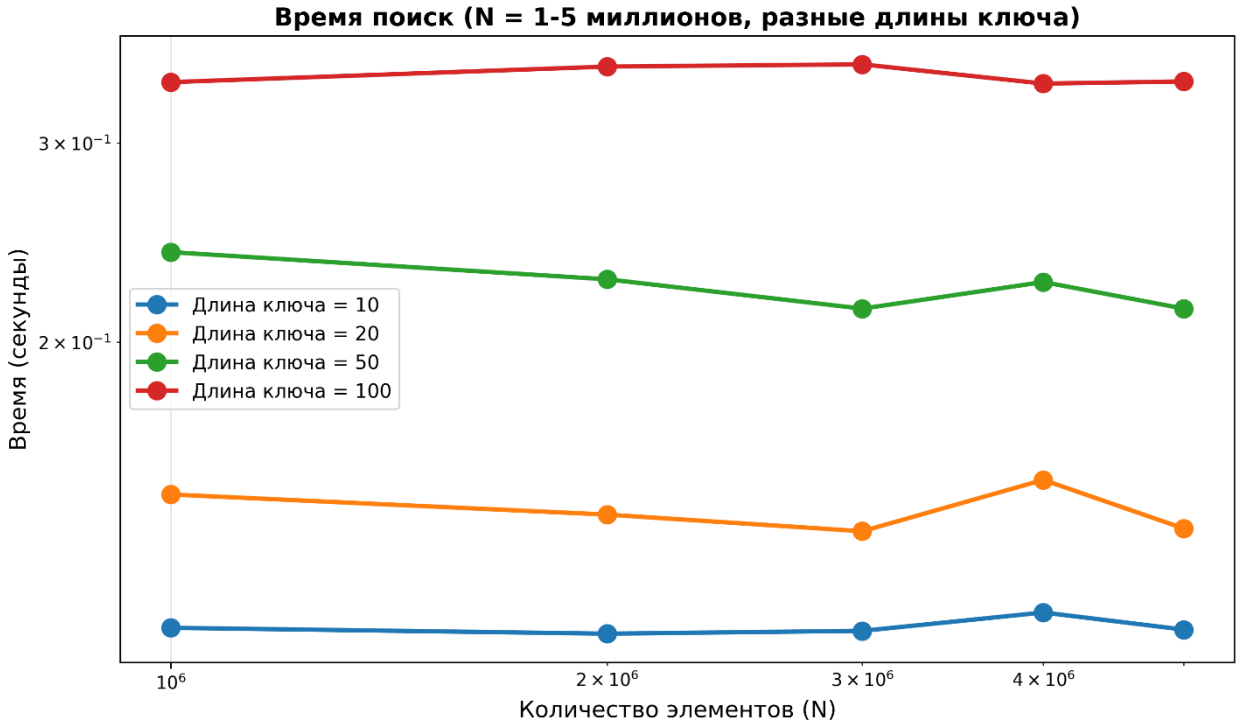
Также был проведён замер зависимости скорости выполнения с фиксированным числом элементов (200000), но с растущем длины ключа (от 10 до 256 символов). Примечание: замер поиска происходил при загрузке туда элементов, поэтому график зависимости поиска, также имеет зависимость

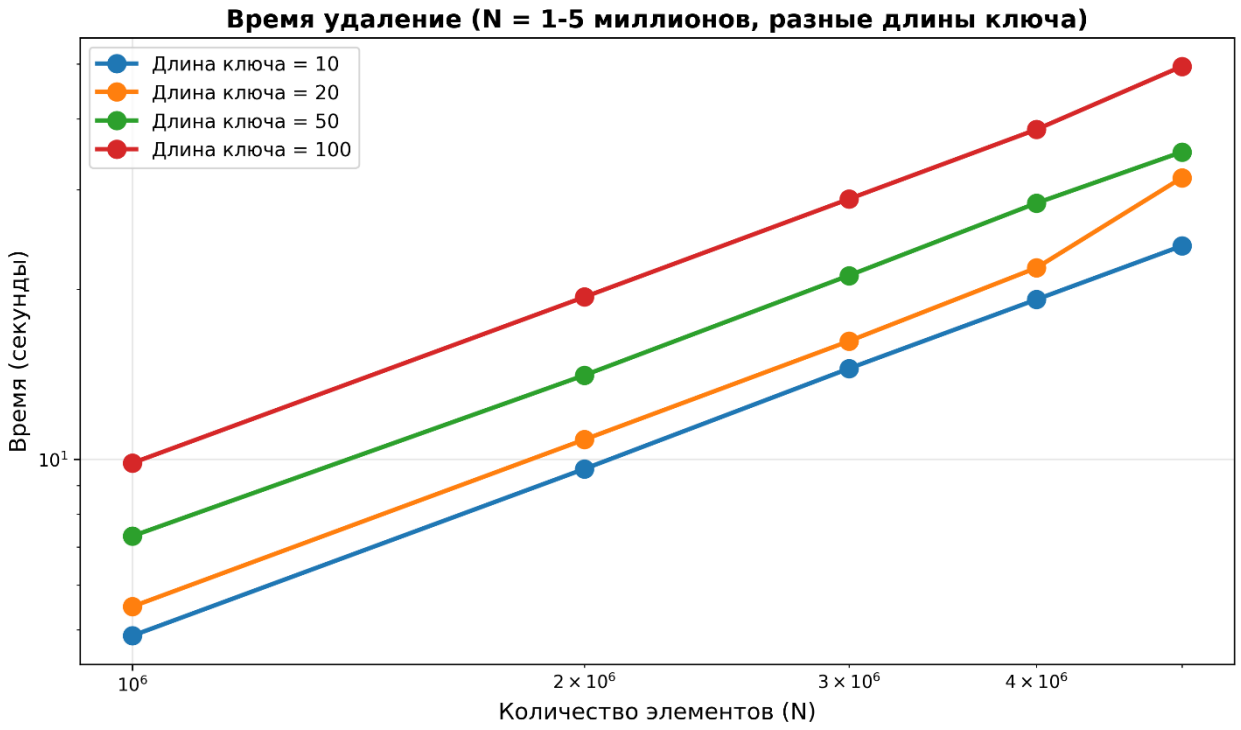




Стресс тест на огромное число элементов (От 1 миллиона до 5, с постепенным просто длины ключа) подтвердил результат:


Время вставка (N = 1-5 миллионов, разные длины ключа)
Длина ключа = 10
Длина ключа = 20
Длина ключа = 50
Длина ключа = 100
Время (секунды)
10^1
10^6
2×10^6
3×10^6
4×10^6
Количество элементов (N)


Время поиск (N = 1-5 миллионов, разные длины ключа)
3×10^{-1}
2×10^{-1}
Длина ключа = 10
Длина ключа = 20
Длина ключа = 50
Длина ключа = 100
Время (секунды)
10^6
2×10^6
3×10^6
4×10^6
Количество элементов (N)



Также ниже приведены тесты с проверяющей решение платформы:

№	Вердикт	Ресурсы	Баллы
1	ok	3ms / 2.17Mb	—
2	ok	3ms / 2.00Mb	—
3	ok	3ms / 2.00Mb	—
4	ok	3ms / 2.13Mb	—
5	ok	200ms / 264.00Kb	—
6	ok	187ms / 35.06Mb	—
7	ok	3ms / 2.12Mb	—
8	ok	3ms / 2.12Mb	—
9	ok	4ms / 2.06Mb	—
10	ok	3ms / 2.12Mb	—
11	ok	4ms / 2.17Mb	—
12	ok	176ms / 28.11Mb	—
13	ok	3ms / 2.00Mb	—
14	ok	0.801s / 16.89Mb	—
15	ok	2.972s / 223.01Mb	—
16	ok	0.586s / 21.40Mb	—
17	ok	199ms / 264.00Kb	—
18	ok	3ms / 2.03Mb	—

Можем заметить, что тесты и графики демонстрируют линейную сложность, по времени:

Сложность по времени – $O(K)$, где K – длинна ключа, откуда следует: Сложность по памяти: $O(N)$, где N – количество элементов?

Недочёты

На мой взгляд недочётом данного алгоритма является сложность его отладки, так как необходимо следить за обратными ссылками (указатель может вести как вниз к потомку, так и вверх к предку), это значительно усложняет процесс визуализации и поиска ошибок доступа к памяти.

Также на мой взгляд недостатком является размещение узлов на куче, а не на выделенном на куче участке памяти, например, на векторе. Это позволит ускорить работу алгоритма и не думать о ручном управлении памятью.

Выводы

В результате выполнения данной лабораторной работы, было успешно реализовано дерево PATRICIA. В ходе выполнения были детально изучены алгоритмы вставки и удаления узлов, а также особенности сериализации шаблонных данных. Проведенные тесты производительности подтвердили, что сложность основных операций (поиск, вставка, удаление) пропорциональна длине ключа, что обеспечивает высокую эффективность на больших объемах данных.

Главными проблемами в процессе разработки стали отладка краевых случаев удаления и реализация корректной записи шаблонных данных в бинарный файл. Все возникшие проблемы были успешно устранены.

На мой взгляд данная программа может служить для огромного спектра задач, где используется большой объём строковых данных и частая потребность поиска. PATRICIA позволяет выполнять операции над огромным количеством элементов с алфавитно-цифровой информацией за разумное время.